

AI-Integrated Intelligent Metro Information System (KMRL)

21CSC205P Database Management Systems
A MINI PROJECT REPORT

Submitted by

Saundarya Singh [RA2411003011364]
Nehal Agarwal [RA2411003011381]

Under the Guidance of

(GUIDE NAME)

(Designation, Department)

*in partial fulfillment of the requirements for the degree
of*

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in **(CSE CORE)**



DEPARTMENT OF NETWORKING AND COMMUNICATIONS
SCHOOL OF COMPUTING
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

MAY 2026



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that Project report titled “**PROJECT TITLE** ” is the bonafide work of “**STUDENT1 NAME [REG NUM], STUDENT2 NAME [REG NUM]**” who carried out the **21CSC205P Database Management Systems** mini project work under my supervision.

COURSE FACULTY
DESIGNATION
Department

HEAD OF THE DEPARMENT
PROFESSOR AND HEAD
Department

ABSTRACT

TABLE OF CONTENTS

Chapter No.	Title	Page no.
	ABSTRACT	
1	Problem understanding, Identification of Entity and Relationships, Construction of DB using ER Model for their project	
2	Design of Relational Schemas, Creation of Database and Tables for their project	
	3.1 Writing queries based on the concepts of Aggregate functions, constraints, sets	
3	3.2 Writing complex queries based on subqueries, joins and views	
	3.3 Writing complex queries based on the concepts of functions, Triggers, Cursors and Exception Handling	
	4.1 Analysing the pitfalls, identifying the dependencies, and applying Normalizations- 1NF, 2NF, 3NF	
4	4.2 Analysing the pitfalls, identifying the dependencies, and applying Normalizations- 4NF, 5NF	
5	Implementation of concurrency control and recovery mechanisms.	
6	Front-end And Back-End Implementation code	
7	Results and Discussions	

LIST OF FIGURES

Figure No.	Name	Page no.
1		
2		
3		
4		
5		

LIST OF TABLES

Figure No.	Name	Page no.
1		
2		
3		
4		
5		

CHAPTER 1

PROBLEM UNDERSTANDING, IDENTIFICATION OF ENTITY AND RELATIONSHIPS, CONSTRUCTION OF DB USING ER MODEL FOR THE AI-Integrated Intelligent Metro Information System (KMRL)

1.1 Introduction

Modern metro transportation systems generate a large volume of structured data such as station details, routes, schedules, fares, passenger information, and operational rules. Efficient storage and retrieval of this data is essential for ensuring smooth functioning and enhancing passenger convenience.

This project presents an AI-Integrated Intelligent Metro Information System designed for Kochi Metro Rail Limited (KMRL). The system is primarily database-driven, where a well-structured relational database forms the core of the application.

To enhance user interaction, the system integrates Retrieval-Augmented Generation (RAG), where user queries are processed through an AI interface that retrieves relevant data from the database and generates meaningful responses. However, the correctness and efficiency of the system depend entirely on the underlying database design.

1.2 Motivation

In real-world metro systems, passengers often face challenges such as difficulty in accessing accurate route information, fare details, and train schedules. Traditional systems may lack efficient query mechanisms or user-friendly interfaces.

The motivation behind this project is to design a robust database system that:

- Eliminates data redundancy
- Ensures data consistency and integrity
- Enables fast and efficient data retrieval
- Provides intelligent query handling using AI

By combining database systems with AI-based query interaction (RAG), the system aims to bridge the gap between structured data and user-friendly information access.

1.3 Scope

The scope of this project includes the design and implementation of a metro information system with the following capabilities:

- Management of metro lines and stations
- Representation of routes between stations
- Scheduling of train arrival and departure times
- Fare calculation between source and destination
- Passenger and ticket management
- Employee and station facility management
- AI-based query interaction using RAG

The project focuses mainly on the database layer, while AI is used as an interface to improve usability.

1.4 Problem Statement

The objective of this project is to design and implement a relational database system for Kochi Metro Rail Limited that:

- Stores structured metro data such as stations, routes, schedules, fares, and passengers
- Maintains data consistency and referential integrity using constraints
- Eliminates redundancy through normalization techniques
- Enables efficient data retrieval using SQL queries
- Integrates RAG-based AI to allow users to query the system in a natural and interactive way

This system aims to demonstrate strong database modeling skills while solving real-world transportation data management problems.

1.5 Project Requirements

Hardware Requirements:

- Computer system with minimum 4GB RAM

Software Requirements:

- MySQL Workbench

- SQL

- Programming environment for AI integration

Technical Requirements:

- Knowledge of DBMS concepts

- Understanding of ER diagrams

- SQL query writing

- Basic understanding of AI and RAG

1.6 Identification of Entity and Relationships

1.6.1 Entities Identified

Metro_Line

Station

Train

Route

Schedule

Fare

Passenger

Ticket (Weak Entity)

Employee

Station_Facility (Multivalued Attribute)

1.6.2 Relationships Identified

One Metro_Line → Many Stations (1:N)

One Metro_Line → Many Trains (1:N)

One Route → Many Schedules (1:N)

Passenger → Ticket (1:N)

Station → Facilities (1:N)

1.6.3 Types of Entities

Strong Entities: Metro_Line, Station, Train, Passenger

Weak Entity: Ticket

Multivalued Attribute: Station_Facility

1.7 Construction of DB Using ER Model for the AI-Integrated Intelligent Metro Information System (KMRL)

The ER model is constructed using the identified entities, attributes, and relationships to represent the structure of the metro system.

The ER diagram visually illustrates:

Entities and their attributes

Relationships between entities

Cardinality constraints

Participation constraints

This model ensures that the database is well-structured and can be efficiently converted into relational schemas.

Chapter 2

Design of Relational Schemas, Creation of Database and Tables for AI-Integrated Intelligent Metro Information System (KMRL)

2.1 Relational Schema for AI-Integrated Intelligent Metro Information System (KMRL)

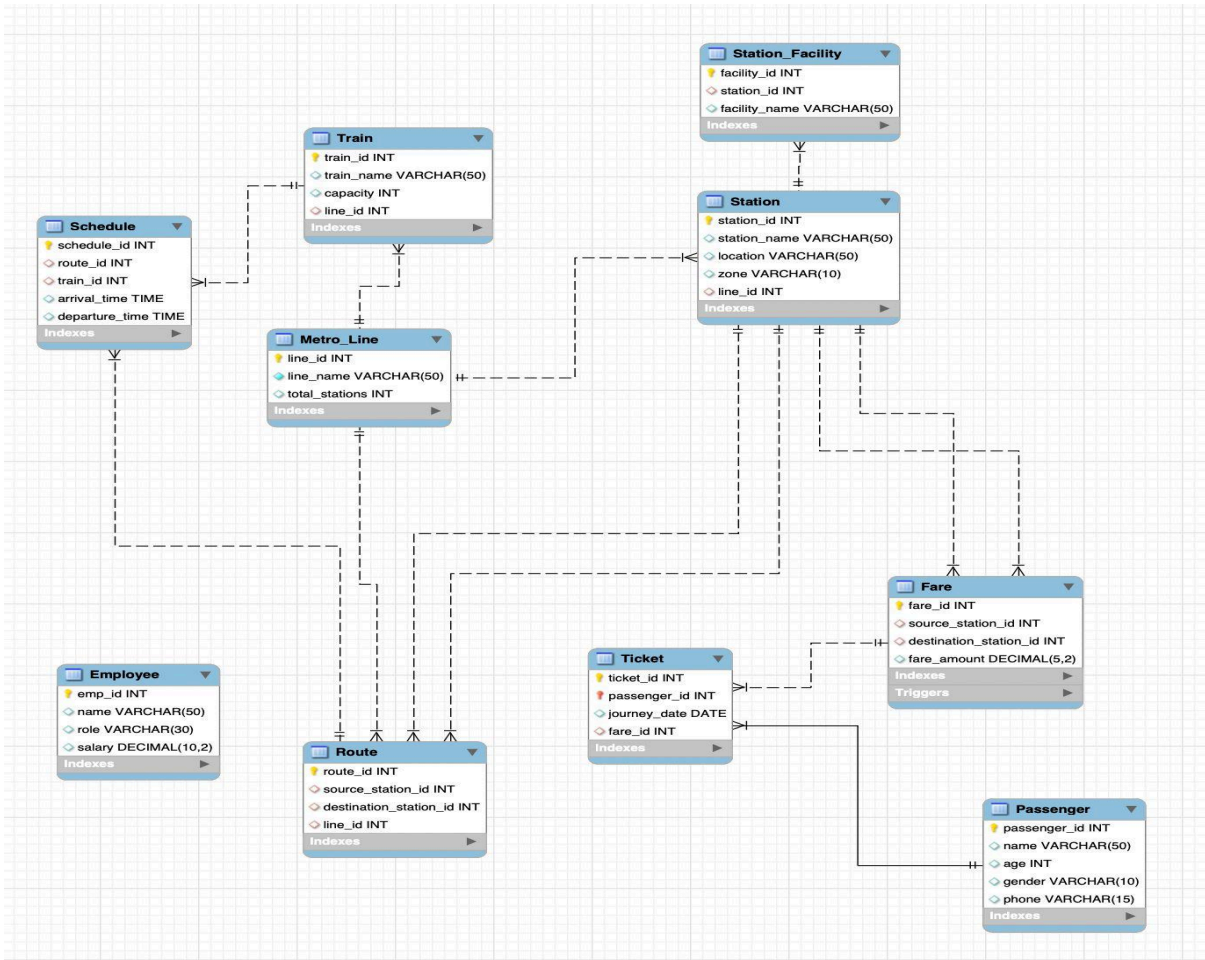


Figure 2.1: Relational Schema for <project title>

Figure 2.1 depicts the relational schema for the AI-Integrated Intelligent Metro Information System.

The schema represents the structure of the database and illustrates how different entities are connected through relationships using primary keys and foreign keys.

The database consists of multiple tables including Metro_Line, Station, Train, Route, Schedule, Fare, Passenger, Ticket, Employee, and Station_Facility, each representing a real-world component of the metro system.

The Metro_Line table stores information about different metro lines and is linked to the Station and Train tables through the line_id foreign key, representing a one-to-many relationship where one metro line can have multiple stations and trains.

The Station table stores details of metro stations such as name, location, and zone. It is connected to multiple tables including Route, Fare, and Station_Facility, indicating that a station can be part of multiple routes, fare calculations, and can have multiple facilities.

The Train table contains details about trains operating on specific metro lines and is linked to the Schedule table, showing that each train can have multiple schedules.

The Route table represents the path between a source station and a destination station. It establishes relationships between stations and metro lines using foreign keys.

The Schedule table stores arrival and departure timings of trains for specific routes, representing a one-to-many relationship between routes and schedules.

The Fare table defines the fare between two stations and resolves the many-to-many relationship between stations by storing source and destination station IDs.

The Passenger table stores passenger details, and the Ticket table (a weak entity) stores ticket booking information. The Ticket table depends on the Passenger table and establishes a relationship between passengers and fare details.

The Employee table stores metro staff information, while the Station_Facility table represents facilities available at stations, indicating a multivalued relationship.

Overall, the relational schema ensures data integrity, reduces redundancy, and enables efficient querying by properly defining relationships between entities using primary and foreign keys.

2.2 Description of Tables

Table 2.1: Metro_Line

1. **Purpose:** Stores details of metro lines.
2. **Primary Key:** line_id (INT)
3. **Data Types and Constraints:**
 - line_id: INT, Primary Key, Not Null

- line_name: VARCHAR(50), Not Null
- total_stations: INT, CHECK (total_stations > 0)

Table 2.2: Station

- **Purpose:** Stores details of metro stations.
- **Primary Key:** station_id (INT)
- **Foreign Key:** line_id → Metro_Line(line_id)
- **Data Types and Constraints:**
 - station_id: INT, Primary Key, Not Null
 - station_name: VARCHAR(50)
 - location: VARCHAR(50)
 - zone: VARCHAR(10)
 - line_id: INT, Foreign Key

Table 2.3: Train

- **Purpose:** Stores train details.
- **Primary Key:** train_id (INTEGER)
- **Data Types and Constraints:**
 - train_id: INT, Primary Key, Not Null
 - train_name: VARCHAR(50)
 - capacity: INT, Check (capacity > 0)
 - line_id: INT, Foreign Key

Table 2.4: Route

- **Purpose:** Stores route information between stations.
- **Primary Key:** route_id (INTEGER)
- **Data Types and Constraints:**
 - route_id: INT, Primary Key
 - source_station_id: INT, Foreign Key
 - destination_station_id: INT, Foreign Key
 - line_id: INT, Foreign Key

Table 2.5: Schedule

- **Purpose:** Stores train timings.
- **Primary Key:** schedule_id (INTEGER)
- **Data Types and Constraints:**
 - schedule_id: INT, Primary Key
 - route_id: INT, Foreign Key
 - train_id: INT, Foreign Key
 - arrival_time: TIME
 - departure_time: TIME

Table 2.6: Fare

- **Purpose:** Stores fare between stations.
- **Primary Key:** fare_id (INTEGER)
- **Data Types and Constraints:**
 - fare_id: INT, Primary Key
 - source_station_id: INT, Foreign Key
 - destination_station_id: INT, Foreign Key
 - fare_amount: DECIMAL(5,2), Check (fare_amount > 0)

Table 2.7: Passenger

- **Purpose:** Stores passenger details.
- **Primary Key:** passenger_id (INTEGER)
- **Data Types and Constraints:**
 - passenger_id: INT, Primary Key
 - name: VARCHAR(50)
 - age: INT, Check (age > 0)
 - gender: VARCHAR(10)
 - phone: VARCHAR(15), Unique

Table 2.8: Ticket (Weak Entity)

- **Purpose:** Stores ticket booking details.
- **Primary Key:** (ticket_id, passenger_id)
- **Data Types and Constraints:**
 - ticket_id: INT
 - passenger_id: INT, Foreign Key
 - journey_date: DATE
 - fare_id: INT, Foreign Key

Table 2.9: Employee

- **Purpose:** Stores employee details.
- **Primary Key:** emp_id (INTEGER)
- **Data Types and Constraints:**
 - emp_id: INT, Primary Key
 - name: VARCHAR(50)
 - role: VARCHAR(30)
 - salary: DECIMAL(10,2), Check (salary > 0)

Table 2.10: Station_Facility

- **Purpose:** Stores facilities available at stations.
- **Primary Key:** facility_id (INTEGER)
- **Data Types and Constraints:**
 - facility_id: INT, Primary Key
 - station_id: INT, Foreign Key
 - facility_name: VARCHAR(50)

2.3 Creation of Database and Tables - DDL Commands

```
DROP DATABASE IF EXISTS MetroDB;
```

```
CREATE DATABASE MetroDB;
```

```
USE MetroDB;
```

```
CREATE TABLE Metro_Line(  
    line_id INT PRIMARY KEY,  
    line_name VARCHAR(50) NOT NULL,  
    total_stations INT CHECK(total_stations > 0)  
);
```

```
CREATE TABLE Station(  
    station_id INT PRIMARY KEY,  
    station_name VARCHAR(50),  
    location VARCHAR(50),  
    zone VARCHAR(10),  
    line_id INT,  
    FOREIGN KEY(line_id) REFERENCES Metro_Line(line_id)  
);
```

```
CREATE TABLE Train(  
    train_id INT PRIMARY KEY,  
    train_name VARCHAR(50),  
    capacity INT CHECK(capacity > 0),  
    line_id INT,  
    FOREIGN KEY(line_id) REFERENCES Metro_Line(line_id)  
);
```

```
CREATE TABLE Route(  
    route_id INT PRIMARY KEY,  
    source_station_id INT,  
    destination_station_id INT,  
    line_id INT,  
    FOREIGN KEY(source_station_id) REFERENCES Station(station_id),  
    FOREIGN KEY(destination_station_id) REFERENCES Station(station_id),  
    FOREIGN KEY(line_id) REFERENCES Metro_Line(line_id)  
);
```

```
CREATE TABLE Schedule(  
    schedule_id INT PRIMARY KEY,  
    route_id INT,  
    train_id INT,  
    arrival_time TIME,  
    departure_time TIME,  
    FOREIGN KEY(route_id) REFERENCES Route(route_id),  
    FOREIGN KEY(train_id) REFERENCES Train(train_id)  
);
```

```
CREATE TABLE Fare(  
    fare_id INT PRIMARY KEY,  
    source_station_id INT,  
    destination_station_id INT,  
    fare_amount DECIMAL(5,2) CHECK(fare_amount > 0),  
    FOREIGN KEY(source_station_id) REFERENCES Station(station_id),  
    FOREIGN KEY(destination_station_id) REFERENCES Station(station_id)  
);
```

```
CREATE TABLE Passenger(  
    passenger_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    age INT CHECK(age > 0),  
    gender VARCHAR(10),  
    phone VARCHAR(15) UNIQUE  
);
```

```
CREATE TABLE Ticket(  
    ticket_id INT,  
    passenger_id INT,  
    journey_date DATE,  
    fare_id INT,  
    PRIMARY KEY(ticket_id, passenger_id),  
    FOREIGN KEY(passenger_id) REFERENCES Passenger(passenger_id),  
    FOREIGN KEY(fare_id) REFERENCES Fare(fare_id)  
);
```

```
CREATE TABLE Employee(  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    role VARCHAR(30),  
    salary DECIMAL(10,2) CHECK(salary > 0)  
);
```

```
CREATE TABLE Station_Facility(  
    facility_id INT PRIMARY KEY,  
    station_id INT,  
    facility_name VARCHAR(50),
```



```
FOREIGN KEY(station_id) REFERENCES Station(station_id)
);
```

2.4 Insertion of tuples into the table – DML commands

Insert SQL (for all tables)

```
INSERT INTO Metro_Line VALUES (1,'Red Line',15),(2,'Blue Line',15),(3,'Green Line',15);
```

Metro_Line table:

line_id	line_name	total_stations
1	Red Line	15
2	Blue Line	15
3	Green Line	15

```
INSERT INTO Station VALUES (1,'Aluva','Kochi','A',1), (2,'Pulinchodu','Kochi','A',1),
(3,'Companypady','Kochi','B',1);
```

Station table:

station_id	station_name	location	zone	line_id
1	Aluva	Kochi	A	1
2	Pulinchodu	Kochi	B	1
3	Companypady	Kochi	B	1

```
INSERT INTO Train VALUES (1,'TrainA',200,1), (2,'TrainB',180,1), (3,'TrainC',220,2);
```

Train table:

train_id	train_name	capacity	line_id
1	TrainA	200	1
2	TrainB	180	1
3	TrainC	220	2

INSERT INTO Passenger VALUES (1,'Rahul',21,'M','9999990001'),
(2,'Riya',22,'F','9999990002'), (3,'Aman',23,'M','9999990003');

Passenger table:

passenger_id	name	age	gender	phone
1	Rahul	21	M	9999990001
2	Riya	22	F	9999990002
3	Aman	23	M	9999990003

INSERT INTO Route VALUES (1,1,5,1),(2,2,6,1),(3,3,7,1);

Route table:

route_id	source_station_id	destination_station_id	line_id
1	1	5	1
2	2	6	1
3	3	7	1

INSERT INTO Schedule VALUES (1,1,1,'08:00:00','08:05:00'), (2,2,2,'08:10:00','08:15:00'), (3,3,3,'08:20:00','08:25:00');

Schedule table:

schedule_id	route_id	train_id	arrival_time	departure_time
1	1	1	08:00:00	08:05:00
2	2	2	08:10:00	08:15:00
3	3	3	08:20:00	08:25:00

INSERT INTO Fare VALUES (1,1,5,20),(2,2,6,25),(3,3,7,30);

Fare table:

fare_id	source_station_id	destination_station_id	fare_amount
1	1	5	20
2	2	6	25
3	3	7	30

INSERT INTO Ticket VALUES (1,1,'2026-03-01',1), (2,2,'2026-03-02',2), (3,3,'2026-03-03',3);

Ticket table:

ticket_id	passenger_id	journey_date	fare_id
1	1	2026-03-0	1
2	2	2026-03-02	2
3	3	2026-03-03	3

INSERT INTO Employee VALUES (1,'Ravi','Manager',50000), (2,'Sita','Staff',30000), (3,'John','Security',25000);

Employee table:

emp_id	name	role	salary
1	Ravi	Manager	50000
2	Sita	Staff	30000
3	John	Security	25000

INSERT INTO Station_Facility VALUES (1,1,'Lift'), (2,1,'Escalator'), (3,2,'Parking');

Station_Facility table:

facility_id	station_id	facility_name
1	1	Lift
2	1	Escalator
3	2	Parking

CHAPTER 3

Complex queries based on the concepts of constraints, sets, joins, views, Triggers and Cursors.

(min 3 questions with queries for all topics)

< create complex queries based on the sub-topic with questions and SQL statement with output>

3.1 Adding Constraints and queries based on constraints

Question: 1

A user registers successfully by entering valid details in the metro system.

SQL Statement:

```
INSERT INTO Passenger VALUES (16,'TestUser',22,'M','9999990020');
```

Output:

```
26 23:35:56 INSERT INTO Passenger VALUES (16,'Tes... 1 row(s) affected 0.0020 sec
```

Question: 2

A user mistakenly enters an invalid age while registering.

SQL Statement:

```
INSERT INTO Passenger VALUES (17,'InvalidUser',-5,'F','9999990021');
```

Output:

```
27 23:35:56 INSERT INTO Passenger VALUES (17,'Inv... Error Code: 3819. Check constraint 'passenger_chk_1... 0.0018 sec
```

Question: 3

A user tries to register using a phone number that already exists in the system.

SQL Statement:

```
INSERT INTO Passenger VALUES (18,'DuplicateUser',25,'M','9999990001');
```

Output:

```
53 08:06:44 INSERT INTO Passenger VALUES (18,'Du... Error Code: 1062. Duplicate entry '9999990001' for k... 0.044 sec
```

3.2 Queries based on Aggregate Functions

Question: 1

The metro authority wants to know the total number of passengers registered in the system.

SQL Statement:

```
SELECT COUNT(*) AS Total_Passengers FROM Passenger;
```

Output:

Total_Passengers
16

Question: 2

The system calculates the average fare between stations.

SQL Statement:

```
SELECT AVG(fare_amount) AS Avg_Fare FROM Fare;
```

Output:

Avg_Fare
55.000000

Question: 3

The metro authority wants to find how many stations are present in each zone.

SQL Statement:

```
SELECT zone, COUNT(*) AS Station_Count  
FROM Station  
GROUP BY zone  
HAVING COUNT(*) > 2;
```

Output:

zone	Station_Count
A	6
B	5
C	4

3.3 Complex queries based on Sets

Question: 1

The system displays a combined list of station names and available facilities without duplicates.

SQL Statement:

```
SELECT station_name FROM Station  
UNION  
SELECT facility_name FROM Station_Facility;
```

Output:

Question: 2

The system displays all station names and facilities including duplicates.

SQL Statement:

```
SELECT station_name FROM Station
UNION ALL
SELECT facility_name FROM Station_Facility;
```

Output:

Question: 3

The system identifies stations that are not used as source stations in fare calculations.

SQL Statement:

```
SELECT station_id FROM Station
WHERE station_id NOT IN (
    SELECT source_station_id FROM Fare
);
```

Output:

3.4 Complex queries based on Subqueries

Question: 1

The metro system identifies fares that are higher than the average fare.

SQL Statement:

```
SELECT * FROM Fare
WHERE fare_amount > (
    SELECT AVG(fare_amount) FROM Fare
);
```

Output:

Question: 2

The system retrieves passengers who have booked tickets.

SQL Statement:

```
SELECT * FROM Passenger
WHERE passenger_id IN (
    SELECT passenger_id FROM Ticket
);
```

Output:**Question: 3**

The system identifies passengers whose age is greater than the average age.

SQL Statement:

```
SELECT * FROM Passenger
WHERE age > (
    SELECT AVG(age) FROM Passenger
);
```

Output:

3.5 Complex queries based on Joins

Question: 1

The system displays passenger names along with their ticket IDs.

SQL Statement:

```
SELECT p.name, t.ticket_id
FROM Passenger p
INNER JOIN Ticket t
ON p.passenger_id = t.passenger_id;
```

Output:**Question: 2**

The system shows all stations along with their fare details if available.

SQL Statement:

```
SELECT s.station_name, f.fare_amount
FROM Station s
LEFT JOIN Fare f
ON s.station_id = f.source_station_id;
```


Output:

Question: 3

The system finds pairs of stations belonging to the same metro line.

SQL Statement:

```
SELECT A.station_name, B.station_name  
FROM Station A, Station B  
WHERE A.line_id = B.line_id  
AND A.station_id <> B.station_id;
```

Output:

3.6 Complex queries based on views

Question: 1

The system creates a view to store high fare records.

SQL Statement:

```
CREATE VIEW HighFare AS  
SELECT * FROM Fare WHERE fare_amount > 50;
```

Output:

Question: 2

The system displays all records from the HighFare view.

SQL Statement:

```
SELECT * FROM HighFare;
```

Output:

Question: 3

The system deletes the HighFare view after usage.

SQL Statement:

```
DROP VIEW HighFare;
```

Output:

3.7 Complex queries based on Triggers

Question: 1

The system creates a trigger to prevent insertion of invalid fare values.

SQL Statement:

```
DELIMITER $$

CREATE TRIGGER check_fare
BEFORE INSERT ON Fare
FOR EACH ROW
BEGIN
    IF NEW.fare_amount < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Invalid Fare';
    END IF;
END$$

DELIMITER ;
```

Output:

Question: 2

A user tries to insert a negative fare value.

SQL Statement:

```
INSERT INTO Fare VALUES (20,1,2,-10);
```

Output:

Question: 3

A valid fare value is inserted into the system.

SQL Statement:

```
INSERT INTO Fare VALUES (21,1,2,25);
```

Output:

3.8 Complex queries based on Cursors

Question: 1

The system creates a procedure to display all passengers.

SQL Statement:

```
DELIMITER $$
```

```
CREATE PROCEDURE GetPassengers()  
BEGIN  
    SELECT * FROM Passenger;  
END$$
```

```
DELIMITER ;
```

Output:

Question: 2

The system executes the procedure to display passenger details.

SQL Statement:

```
CALL GetPassengers();
```

Output:

Question: 3

The system uses a cursor to iterate through passenger names.

SQL Statement:

```
DELIMITER $$
```

```
CREATE PROCEDURE CursorExample()  
BEGIN  
    DECLARE done INT DEFAULT FALSE;  
    DECLARE pname VARCHAR(50);
```

```
    DECLARE cur CURSOR FOR SELECT name FROM Passenger;  
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
```

```
    OPEN cur;
```

```
read_loop: LOOP
  FETCH cur INTO pname;
  IF done THEN
    LEAVE read_loop;
  END IF;
END LOOP;

CLOSE cur;
END$$

DELIMITER ;
```

Output:

CHAPTER 4

ANALYZING THE PITFALLS, IDENTIFYING THE DEPENDENCIES, AND APPLYING NORMALIZATIONS

4.1 Analyse the Pitfalls in Relations

Before normalization, data is stored in a single table combining passenger, station, train, and fare details. This leads to:

- Redundancy → Same station/train repeated
- Insertion anomaly → Cannot insert station without passenger
- Deletion anomaly → Deleting ticket deletes passenger info
- Update anomaly → Updating station name requires multiple changes

SQL (Unnormalized Table)

```
CREATE TABLE Metro_Unnormalized (  
    passenger_id INT,  
    passenger_name VARCHAR(50),  
    phone VARCHAR(15),  
    station_name VARCHAR(50),  
    facilities VARCHAR(100),  
    route_id INT,  
    fare_amount DECIMAL(5,2),  
    train_name VARCHAR(50),  
    schedule_time TIME  
);
```

```

16 INSERT INTO Metro_UNF VALUES
17 (1,'Rahul','9999','Aluva','Lift,Escalator','TrainA','Aluva-Pulinchodu',20),
18 (2,'Rahul','9999','Aluva','Lift,Escalator','TrainB','Aluva-Companypady',25),
19 (3,'Riya','8888','Pulinchodu','Parking','TrainA','Pulinchodu-Aluva',20);
20
21 SELECT * FROM Metro_UNF;
22
23

```

1:23

Result Grid Filter Rows: Search Export:

ticket_id	passenger_name	phone	station_name	facilities	train_name	route	fare
1	Rahul	9999	Aluva	Lift,Escalator	TrainA	Aluva-Pulinchodu	20
2	Rahul	9999	Aluva	Lift,Escalator	TrainB	Aluva-Companypady	25
3	Riya	8888	Pulinchodu	Parking	TrainA	Pulinchodu-Aluva	20

4.2 First Normal Form

BEFORE 1NF

```

16 INSERT INTO Metro_UNF VALUES
17 (1,'Rahul','9999','Aluva','Lift,Escalator','TrainA','Aluva-Pulinchodu',20),
18 (2,'Rahul','9999','Aluva','Lift,Escalator','TrainB','Aluva-Companypady',25),
19 (3,'Riya','8888','Pulinchodu','Parking','TrainA','Pulinchodu-Aluva',20);
20
21 SELECT * FROM Metro_UNF;
22
23

```

1:23

Result Grid Filter Rows: Search Export:

ticket_id	passenger_name	phone	station_name	facilities	train_name	route	fare
1	Rahul	9999	Aluva	Lift,Escalator	TrainA	Aluva-Pulinchodu	20
2	Rahul	9999	Aluva	Lift,Escalator	TrainB	Aluva-Companypady	25
3	Riya	8888	Pulinchodu	Parking	TrainA	Pulinchodu-Aluva	20

4.2.1: Identify Dependency

- ticket_id → passenger_name, phone, station_name, train_name, route, fare
- station_name → facilities (multi-valued dependency)

4.2.2: Apply Normalization to 1NF

```

26 SELECT * FROM Ticket_1NF;
27 SELECT * FROM Station_Facility_1NF;
28
29

```

00% 3:9

Result Grid Filter Rows: Search Export:

	ticket_id	passenger_name	phone	station_name	train_name	route	fare	
	1	Rahul	9999	Aluva	TrainA	Aluva-Pulinchodu	20	
	2	Rahul	9999	Aluva	TrainB	Aluva-Companypady	25	
	3	Riya	8888	Pulinchodu	TrainA	Pulinchodu-Aluva	20	

Ticket_1NF 1 Station_Facility_1NF 2

```

26 SELECT * FROM Ticket_1NF;
27 SELECT * FROM Station_Facility_1NF;
28
29

```

00% 1:29

Result Grid Filter Rows: Search Export:

	station_name	facility	
	Aluva	Lift	
	Aluva	Escalator	
	Pulinchodu	Parking	

4.3 Second Normal Form

BEFORE 2NF AFTER 1NF

```

26 SELECT * FROM Ticket_1NF;
27 SELECT * FROM Station_Facility_1NF;
28
29

```

00% 3:9

Result Grid Filter Rows: Search Export:

	ticket_id	passenger_name	phone	station_name	train_name	route	fare	
	1	Rahul	9999	Aluva	TrainA	Aluva-Pulinchodu	20	
	2	Rahul	9999	Aluva	TrainB	Aluva-Comanypady	25	
	3	Riya	8888	Pulinchodu	TrainA	Pulinchodu-Aluva	20	

Ticket_1NF 1 Station_Facility_1NF 2

```

26 SELECT * FROM Ticket_1NF;
27 SELECT * FROM Station_Facility_1NF;
28
29

```

00% 1:29

Result Grid Filter Rows: Search Export:

	station_name	facility	
	Aluva	Lift	
	Aluva	Escalator	
	Pulinchodu	Parking	

4.3.1: Identify Dependency

From 1NF:

- ticket_id → passenger_name, phone

4.3.2: Apply Normalization to 2NF


```

22
23 • SELECT * FROM Passenger_2NF;
24 • SELECT * FROM Ticket_2NF;
25

```

00% 31:7

Result Grid Filter Rows: Search Edit: Export/Import:

passenger_id	passenger_name	phone
1	Rahul	9999
2	Riya	8888
NULL	NULL	NULL

```

22
23 • SELECT * FROM Passenger_2NF;
24 • SELECT * FROM Ticket_2NF;
25

```

00% 31:7

Result Grid Filter Rows: Search Edit: Export/Import:

ticket_id	passenger_id	train_name	route	fare
1	1	TrainA	Aluva-Pulinchodu	20
2	1	TrainB	Aluva-Companyady	25
3	2	TrainA	Pulinchodu-Aluva	20
NULL	NULL	NULL	NULL	NULL

4.4 Third Normal Form

BEFORE 3NF AND AFTER 2NF

```

22
23 • SELECT * FROM Passenger_2NF;
24 • SELECT * FROM Ticket_2NF;
25

```

00% 31:7

Result Grid Filter Rows: Search Edit: Export/Import:

passenger_id	passenger_name	phone
1	Rahul	9999
2	Riya	8888
NULL	NULL	NULL

```

22
23 SELECT * FROM Passenger_2NF;
24 SELECT * FROM Ticket_2NF;
25

```

00% 31:7

Result Grid Filter Rows: Search Edit: Export/Import:

ticket_id	passenger_id	train_name	route	fare
1	1	TrainA	Aluva-Pulinchodu	20
2	1	TrainB	Aluva-Companypady	25
3	2	TrainA	Pulinchodu-Aluva	20
NULL	NULL	NULL	NULL	NULL

4.4.1: Identify Dependency

FROM 2NF:

route → source_station, destination_station

ticket_id → route → station

4.4.2: Apply Normalization to 3NF

```

36 SELECT * FROM Station_3NF;
37 SELECT * FROM Route_3NF;
38 SELECT * FROM Ticket_3NF;
39

```

00% 1:41

Result Grid Filter Rows: Search Edit: Export/Import:

station_id	station_name
1	Aluva
2	Pulinchodu
3	Companypady
NULL	NULL

```

36 SELECT * FROM Station_3NF;
37 SELECT * FROM Route_3NF;
38 SELECT * FROM Ticket_3NF;
39

```

00% 1:41

Result Grid Filter Rows: Search Edit: Export/Import:

route_id	source_station	destination_station
1	Aluva	Pulinchodu
2	Aluva	Companypady
3	Pulinchodu	Aluva
NULL	NULL	NULL

```

36 SELECT * FROM Station_3NF;
37 SELECT * FROM Route_3NF;
38 SELECT * FROM Ticket_3NF;
39

```

00% 1:41

Result Grid Filter Rows: Search Edit: Export/Import:

ticket_id	passenger_id	route_id	train_name	fare
1	1	1	TrainA	20
2	1	2	TrainB	25
3	2	3	TrainA	20
NULL	NULL	NULL	NULL	NULL

4.5 BCNF

BEFORE BNF AFTER 3NF

```

36 SELECT * FROM Station_3NF;
37 SELECT * FROM Route_3NF;
38 SELECT * FROM Ticket_3NF;
39

```

00% 1:41

Result Grid Filter Rows: Search Edit: Export/Import:

station_id	station_name
1	Aluva
2	Pulinchodu
3	Companypady
NULL	NULL

```

36 SELECT * FROM Station_3NF;
37 SELECT * FROM Route_3NF;
38 SELECT * FROM Ticket_3NF;
39

```

00% 1:41

Result Grid Filter Rows: Search Edit: Export/Import:

route_id	source_stati...	destination_stati...
1	Aluva	Pulinchodu
2	Aluva	Companypady
3	Pulinchodu	Aluva
NULL	NULL	NULL

```

36 SELECT * FROM Station_3NF;
37 SELECT * FROM Route_3NF;
38 SELECT * FROM Ticket_3NF;
39

```

00% 1:41

Result Grid Filter Rows: Search Edit: Export/Import:

ticket_id	passenger_id	route_id	train_name	fare
1	1	1	TrainA	20
2	1	2	TrainB	25
3	2	3	TrainA	20
NULL	NULL	NULL	NULL	NULL

4.5.1: Identify Dependency

CHECK:

route_id → stations ✓

ticket_id → all attributes ✓

4.5.2: Apply Normalization to BCNF

26:2

Result Grid Filter Rows: Search Edit: Export/Import:

ticket_id	passenger_id	route_id	train_name	fare
1	1	1	TrainA	20
2	1	2	TrainB	25
3	2	3	TrainA	20
NULL	NULL	NULL	NULL	NULL

Ticket_3NF 6

4.6 Fourth Normal Form

```

36 SELECT * FROM Station_3NF;
37 SELECT * FROM Route_3NF;
38 SELECT * FROM Ticket_3NF;
39

```

00% 1:41

Result Grid Filter Rows: Search Edit: Export/Import:

station_id	station_name
1	Aluva
2	Pulinchodu
3	Companypady
NULL	NULL

```

36 SELECT * FROM Station_3NF;
37 SELECT * FROM Route_3NF;
38 SELECT * FROM Ticket_3NF;
39

```

00% 1:41

Result Grid Filter Rows: Search Edit: Export/Import:

route_id	source_station_id	destination_station_id
1	Aluva	Pulinchodu
2	Aluva	Companypady
3	Pulinchodu	Aluva
NULL	NULL	NULL

4.6.1: Identify Dependency

station_id $\twoheadrightarrow$ facility (multi-valued dependency)

4.6.2: Apply Normalization to 4NF

```

12
13 SELECT * FROM Station_Facility_4NF;
14

```

100% 1:15

Result Grid Filter Rows: Search Export:

station_id	facility
1	Lift
1	Escalator
2	Parking

4.7 Fifth Normal Form

BEFORE 5NF AFTER 4NF

```

12
13 SELECT * FROM Station_Facility_4NF;
14

```

100% 1:15

Result Grid Filter Rows: Search Export:

	station_id	facility
	1	Lift
	1	Escalator
	2	Parking

4.7.1: Identify Dependency

(route_id, train_name, station_id)

4.7.2: Apply Normalization to 5NF

```

35 SELECT * FROM Route_Train_5NF;
36 SELECT * FROM Train_Station_5NF;
37 SELECT * FROM Route_Station_5NF;

```

100% 37:27

Result Grid Filter Rows: Search Export:

	route_id	train_name
	1	TrainA
	2	TrainB
	3	TrainA

```

35 SELECT * FROM Route_Train_5NF;
36 SELECT * FROM Train_Station_5NF;
37 SELECT * FROM Route_Station_5NF;

```

100% 37:27

Result Grid Filter Rows: Search Export:

	route_id	station_id	
	1	1	
	1	2	
	2	1	
	2	3	
	3	2	
	3	1	

```

35 SELECT * FROM Route_Train_5NF;
36 SELECT * FROM Train_Station_5NF;
37 SELECT * FROM Route_Station_5NF;

```

100% 37:27

Result Grid Filter Rows: Search Export:

	train_name	station_id	
	TrainA	1	
	TrainA	2	
	TrainB	1	
	TrainB	3	

CHAPTER 5

IMPLEMENTATION OF CONCURRENCY CONTROL AND RECOVERY MECHANISMS

5.1 Introduction to Transactions

A transaction is a sequence of one or more SQL operations executed as a single logical unit of work.

In the metro system database, transactions ensure that operations such as ticket booking, passenger updates, and fare processing are performed reliably.

For example, when a ticket is booked, multiple updates occur. These updates must either complete fully or not happen at all.

5.1.1 Properties

- **Atomicity**

Ensures all operations are completed or none are applied

- **Consistency**

Maintains database integrity

- **Isolation**

Prevents interference between transactions

- **Durability**

Changes remain permanently after commit

5.1.2 States

- a. **Active** – The transaction is currently executing its operations.
- b. **Partially Committed** – The transaction has finished execution but is not yet permanently saved to the database.
- c. **Committed** – The transaction has successfully completed and all changes are permanently stored.
- d. **Failed** – The transaction cannot proceed due to an error or system issue.
- e. **Aborted** – The transaction has been rolled back and all its changes are undone.

5.2 Transaction Control Language (TCL)

Transaction Control Language (TCL) is a set of SQL commands used to manage transactions in a database, ensuring data consistency and integrity.

5.2.1 Save point

Used to rollback to a specific point

5.2.2 Commit

Saves changes permanently

5.2.3 Rollback

Undoes changes

5.3 create 5 transactions for your project and execute

5.3.1 Transaction1 code with output (min 5 transactions)

TRANSACTION 1

-- Step 1: Start transaction

START TRANSACTION;

-- Step 2: Update passenger name

UPDATE Passenger_2NF

SET passenger_name = 'Rahul Updated'

WHERE passenger_id = 1;

-- Step 3: Savepoint A

SAVEPOINT A;

-- Step 4: Insert new passenger

INSERT INTO Passenger_2NF VALUES (3,'Aman','7777');

-- Step 5: Savepoint B

SAVEPOINT B;

-- Step 6: Insert another passenger

INSERT INTO Passenger_2NF VALUES (4,'Neha','6666');

-- Step 7: View table

SELECT * FROM Passenger_2NF;

-- Step 8: Rollback to B

ROLLBACK TO B;

SELECT * FROM Passenger_2NF;

-- Step 9: Rollback to A

ROLLBACK TO A;

SELECT * FROM Passenger_2NF;

-- Step 10: Commit

```
COMMIT;  
SELECT * FROM Passenger_2NF;
```

```
mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)  
  
mysql>  
mysql> UPDATE Passenger_2NF  
-> SET passenger_name = 'Rahul Updated'  
-> WHERE passenger_id = 1;  
Query OK, 1 row affected (0.01 sec)  
Rows matched: 1 Changed: 1 Warnings: 0  
  
mysql>  
mysql> SAVEPOINT A;  
Query OK, 0 rows affected (0.00 sec)  
  
mysql>  
mysql> INSERT INTO Passenger_2NF VALUES (3, 'Aman', '7777');  
Query OK, 1 row affected (0.00 sec)  
  
mysql>  
mysql> SAVEPOINT B;  
Query OK, 0 rows affected (0.00 sec)  
  
mysql>  
mysql> INSERT INTO Passenger_2NF VALUES (4, 'Neha', '6666');  
Query OK, 1 row affected (0.00 sec)  
  
mysql>  
mysql> SELECT * FROM Passenger_2NF;  
+-----+-----+-----+  
| passenger_id | passenger_name | phone |  
+-----+-----+-----+  
|          1 | Rahul Updated | 9999 |  
|          2 | Riya          | 8888 |  
|          3 | Aman          | 7777 |  
|          4 | Neha          | 6666 |  
+-----+-----+-----+  
4 rows in set (0.00 sec)  
  
mysql>  
mysql> ROLLBACK TO B;  
Query OK, 0 rows affected (0.00 sec)  
  
mysql> SELECT * FROM Passenger_2NF;  
+-----+-----+-----+  
| passenger_id | passenger_name | phone |  
+-----+-----+-----+  
|          1 | Rahul Updated | 9999 |  
|          2 | Riya          | 8888 |  
|          3 | Aman          | 7777 |  
+-----+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql>  
mysql> ROLLBACK TO A;  
Query OK, 0 rows affected (0.00 sec)
```

```

mysql>
mysql> ROLLBACK TO A;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM Passenger_2NF;
+-----+-----+-----+
| passenger_id | passenger_name | phone |
+-----+-----+-----+
|          1 | Rahul Updated | 9999 |
|          2 | Riya          | 8888 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM Passenger_2NF;
+-----+-----+-----+
| passenger_id | passenger_name | phone |
+-----+-----+-----+
|          1 | Rahul Updated | 9999 |
|          2 | Riya          | 8888 |
+-----+-----+-----+
2 rows in set (0.00 sec)

```

Transaction 2:

-- Start transaction

START TRANSACTION;

-- Update ticket fare

UPDATE Ticket_2NF

SET fare = 30

WHERE ticket_id = 1;

SAVEPOINT A;

-- Insert new ticket

INSERT INTO Ticket_2NF VALUES (4,1,'TrainC','RouteX',40);

SAVEPOINT B;

-- Wrong insert

INSERT INTO Ticket_2NF VALUES (5,1,'Invalid',NULL,-10);

-- Rollback wrong operation

ROLLBACK TO B;

-- View result

SELECT * FROM Ticket_2NF;

COMMIT;

```
mysql> -- Start transaction
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Update ticket fare
mysql> UPDATE Ticket_2NF
  -> SET fare = 30
  -> WHERE ticket_id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SAVEPOINT A;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Insert new ticket
mysql> INSERT INTO Ticket_2NF VALUES (4,1,'TrainC','RouteX',40);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> SAVEPOINT B;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Wrong insert
mysql> INSERT INTO Ticket_2NF VALUES (5,1,'Invalid',NULL,-10);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> -- Rollback wrong operation
mysql> ROLLBACK TO B;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- View result
mysql> SELECT * FROM Ticket_2NF;
+-----+-----+-----+-----+-----+
| ticket_id | passenger_id | train_name | route | fare |
+-----+-----+-----+-----+-----+
| 1 | 1 | TrainA | Aluva-Pulinchodu | 30 |
| 2 | 1 | TrainB | Aluva-Companypady | 25 |
| 3 | 2 | TrainA | Pulinchodu-Aluva | 20 |
| 4 | 1 | TrainC | RouteX | 40 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

Transaction 3:

-- Start transaction

START TRANSACTION;

-- Update route

UPDATE Route_3NF

```
SET source_station = 'Aluva Updated'
WHERE route_id = 1;

-- View after update
SELECT * FROM Route_3NF;

-- Savepoint A
SAVEPOINT A;

-- Insert route
INSERT INTO Route_3NF VALUES (4,'StationX','StationY');

-- View after insert
SELECT * FROM Route_3NF;

-- Savepoint B
SAVEPOINT B;

-- Wrong update
UPDATE Route_3NF
SET source_station = NULL
WHERE route_id = 1;

-- View before rollback
SELECT * FROM Route_3NF;

-- Rollback
ROLLBACK TO B;

-- View after rollback
SELECT * FROM Route_3NF;

-- Commit
COMMIT;
```

```

mysql> -- Start transaction
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Update route
mysql> UPDATE Route_3NF
    -> SET source_station = 'Aluva Updated'
    -> WHERE route_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> -- View after update
mysql> SELECT * FROM Route_3NF;
+-----+-----+-----+
| route_id | source_station | destination_station |
+-----+-----+-----+
|          1 | Aluva Updated | Pulinchodu          |
|          2 | Aluva          | Companypady          |
|          3 | Pulinchodu     | Aluva                |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- Savepoint A
mysql> SAVEPOINT A;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Insert route
mysql> INSERT INTO Route_3NF VALUES (4,'StationX','StationY');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> -- View after insert
mysql> SELECT * FROM Route_3NF;
+-----+-----+-----+
| route_id | source_station | destination_station |
+-----+-----+-----+
|          1 | Aluva Updated | Pulinchodu          |
|          2 | Aluva          | Companypady          |
|          3 | Pulinchodu     | Aluva                |
|          4 | StationX       | StationY             |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> -- Savepoint B
mysql> SAVEPOINT B;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Wrong update
mysql> UPDATE Route_3NF
    -> SET source_station = NULL

```

```

mysql> -- Wrong update
mysql> UPDATE Route_3NF
    -> SET source_station = NULL
    -> WHERE route_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> -- View before rollback
mysql> SELECT * FROM Route_3NF;
+-----+-----+-----+
| route_id | source_station | destination_station |
+-----+-----+-----+
|      1   | NULL          | Pulinchodu         |
|      2   | Aluva         | Companypady        |
|      3   | Pulinchodu    | Aluva              |
|      4   | StationX      | StationY           |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> -- Rollback
mysql> ROLLBACK TO B;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- View after rollback
mysql> SELECT * FROM Route_3NF;
+-----+-----+-----+
| route_id | source_station | destination_station |
+-----+-----+-----+
|      1   | Aluva Updated | Pulinchodu         |
|      2   | Aluva         | Companypady        |
|      3   | Pulinchodu    | Aluva              |
|      4   | StationX      | StationY           |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> -- Commit
mysql> COMMIT;
Query OK, 0 rows affected (0.02 sec)

mysql> |

```

Transaction 4:

-- Start transaction

START TRANSACTION;

-- Update station

UPDATE Station_3NF

SET station_name = 'NewStation'

```
WHERE station_id = 1;

-- View after update
SELECT * FROM Station_3NF;

-- Savepoint A
SAVEPOINT A;

-- Insert station
INSERT INTO Station_3NF VALUES (4,'TestStation');

-- View after insert
SELECT * FROM Station_3NF;

-- Savepoint B
SAVEPOINT B;

-- Wrong insert
INSERT INTO Station_3NF VALUES (5,NULL);

-- View before rollback
SELECT * FROM Station_3NF;

-- Rollback
ROLLBACK TO B;

-- View after rollback
SELECT * FROM Station_3NF;

-- Commit
COMMIT;
```



```

mysql> -- Start transaction
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Update station
mysql> UPDATE Station_3NF
    -> SET station_name = 'NewStation'
    -> WHERE station_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> -- View after update
mysql> SELECT * FROM Station_3NF;
+-----+-----+
| station_id | station_name |
+-----+-----+
|          1 | NewStation   |
|          2 | Pulinchodu   |
|          3 | Companypady  |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- Savepoint A
mysql> SAVEPOINT A;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Insert station
mysql> INSERT INTO Station_3NF VALUES (4,'TestStation');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> -- View after insert
mysql> SELECT * FROM Station_3NF;
+-----+-----+
| station_id | station_name |
+-----+-----+
|          1 | NewStation   |
|          2 | Pulinchodu   |
|          3 | Companypady  |
|          4 | TestStation  |
+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> -- Savepoint B
mysql> SAVEPOINT B;
Query OK, 0 rows affected (0.00 sec)

```

```

mysql>
mysql> -- Wrong insert
mysql> INSERT INTO Station_3NF VALUES (5,NULL);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> -- View before rollback
mysql> SELECT * FROM Station_3NF;
+-----+-----+
| station_id | station_name |
+-----+-----+
|          1 | NewStation   |
|          2 | Pulinchodu   |
|          3 | Companypady  |
|          4 | TestStation  |
|          5 | NULL         |
+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> -- Rollback
mysql> ROLLBACK TO B;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- View after rollback
mysql> SELECT * FROM Station_3NF;
+-----+-----+
| station_id | station_name |
+-----+-----+
|          1 | NewStation   |
|          2 | Pulinchodu   |
|          3 | Companypady  |
|          4 | TestStation  |
+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> -- Commit
mysql> COMMIT;
Query OK, 0 rows affected (0.02 sec)

```

Transaction 5:

-- Start transaction

START TRANSACTION;

-- Update facility

UPDATE Station_Facility_4NF

SET facility = 'Lift Updated'

WHERE station_id = 1;

```
-- View after update
SELECT * FROM Station_Facility_4NF;

-- Savepoint A
SAVEPOINT A;

-- Insert facility
INSERT INTO Station_Facility_4NF VALUES (3,'WiFi');

-- View after insert
SELECT * FROM Station_Facility_4NF;

-- Savepoint B
SAVEPOINT B;

-- Wrong insert
INSERT INTO Station_Facility_4NF VALUES (3,NULL);

-- View before rollback
SELECT * FROM Station_Facility_4NF;

-- Rollback
ROLLBACK TO B;

-- View after rollback
SELECT * FROM Station_Facility_4NF;

-- Commit
COMMIT;
```

```

mysql> -- Start transaction
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Update facility
mysql> UPDATE Station_Facility_4NF
    -> SET facility = 'Lift Updated'
    -> WHERE station_id = 1;
Query OK, 2 rows affected (0.00 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql>
mysql> -- View after update
mysql> SELECT * FROM Station_Facility_4NF;
+-----+-----+
| station_id | facility |
+-----+-----+
|          1 | Lift Updated |
|          1 | Lift Updated |
|          2 | Parking |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- Savepoint A
mysql> SAVEPOINT A;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Insert facility
mysql> INSERT INTO Station_Facility_4NF VALUES (3,'WiFi');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> -- View after insert
mysql> SELECT * FROM Station_Facility_4NF;
+-----+-----+
| station_id | facility |
+-----+-----+
|          1 | Lift Updated |
|          1 | Lift Updated |
|          2 | Parking |
|          3 | WiFi |
+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> -- Savepoint B
mysql> SAVEPOINT B;
Query OK, 0 rows affected (0.00 sec)

```

```

mysql>
mysql> -- Wrong insert
mysql> INSERT INTO Station_Facility_4NF VALUES (3,NULL);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> -- View before rollback
mysql> SELECT * FROM Station_Facility_4NF;
+-----+-----+
| station_id | facility |
+-----+-----+
|          1 | Lift Updated |
|          1 | Lift Updated |
|          2 | Parking |
|          3 | WiFi |
|          3 | NULL |
+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> -- Rollback
mysql> ROLLBACK TO B;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- View after rollback
mysql> SELECT * FROM Station_Facility_4NF;
+-----+-----+
| station_id | facility |
+-----+-----+
|          1 | Lift Updated |
|          1 | Lift Updated |
|          2 | Parking |
|          3 | WiFi |
+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> -- Commit
mysql> COMMIT;
Query OK, 0 rows affected (0.02 sec)

```

5.3 Concurrency control

Concurrency control ensures that multiple transactions execute safely without conflicts.

5.3.1 Concurrency control Algorithms

1. Lock-Based Protocols

Transactions lock data before accessing it.

2. Timestamp-Based Protocols

Transactions are ordered using timestamps.

3. Optimistic Concurrency Control

Transactions execute without locks and validate before commit.

5.3.1 Locking commands

a. Row-Level Locking –

START TRANSACTION;

-- Lock specific row

```
SELECT * FROM Passenger_2NF  
WHERE passenger_id = 1  
FOR UPDATE;
```

Explanation: Locks only the selected row so other transactions cannot modify it.

b. Table-Level Locking –

```
LOCK TABLES Passenger_2NF WRITE;  
UNLOCK TABLES;
```

Explanation: Locks the entire table to prevent other transactions from accessing it.

Lock Modes

Lock Mode	Description
ROW SHARE	Allows concurrent access; prevents other sessions from locking the table exclusively.
ROW EXCLUSIVE	Prevents other sessions from locking in share mode. Used by default for DML.
SHARE	Allows queries but not updates or deletes.
SHARE ROW EXCLUSIVE	A mix; more restrictive than SHARE.
EXCLUSIVE	Prevents all other access — full table lock.

c. COMMIT – Release All Locks

COMMIT;

Explanation: Saves changes and releases all locks.

d. ROLLBACK – Undo Changes & Release Locks

```
ROLLBACK;
```

Explanation: Cancels changes and releases locks.

5.3.2 Example: Row-Level Lock with Transaction

-- Step 1: Start transaction

```
START TRANSACTION;
```

-- Step 2: Lock a row

```
SELECT * FROM Ticket_2NF  
WHERE ticket_id = 1  
FOR UPDATE;
```

-- Step 3: Update the row

```
UPDATE Ticket_2NF  
SET fare = 50  
WHERE ticket_id = 1;
```

-- Step 4: View updated value

```
SELECT * FROM Ticket_2NF;
```

-- Step 5: Commit changes (release lock)

```
COMMIT;
```

```
SELECT * FROM Ticket_2NF WHERE ticket_id = 1;
```

```

mysql> -- Step 1: Start transaction
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Step 2: Lock a row
mysql> SELECT * FROM Ticket_2NF
    -> WHERE ticket_id = 1
    -> FOR UPDATE;
+-----+-----+-----+-----+-----+
| ticket_id | passenger_id | train_name | route | fare |
+-----+-----+-----+-----+-----+
| 1 | 1 | TrainA | Aluva-Pulinchodu | 30 |
+-----+-----+-----+-----+-----+
1 row in set (0.02 sec)

mysql>
mysql> -- Step 3: Update the row
mysql> UPDATE Ticket_2NF
    -> SET fare = 50
    -> WHERE ticket_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
mysql> -- Step 4: View updated value
mysql> SELECT * FROM Ticket_2NF;
+-----+-----+-----+-----+-----+
| ticket_id | passenger_id | train_name | route | fare |
+-----+-----+-----+-----+-----+
| 1 | 1 | TrainA | Aluva-Pulinchodu | 50 |
| 2 | 1 | TrainB | Aluva-Comanypady | 25 |
| 3 | 2 | TrainA | Pulinchodu-Aluva | 20 |
| 4 | 1 | TrainC | RouteX | 40 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> -- Step 5: Commit changes (release lock)
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

```


CHAPTER 6

FRONT-END AND BACK-END CODE OF <PROJECT TITLE>

6.1 Front–End Module codes

6.2 Database connectivity

CHAPTER 7

RESULTS AND DISCUSSIONS

7.1 Screenshots of front-end

7.2 Screenshots of Database

References